

XMI-Based SysML Model Transformation

Yupeng Li

State Key Lab of CAD/CG
Zhejiang University
Hangzhou, China
HangzhouHope System
Technology
Hangzhou, China
liyvpeng2020@126.com

Hailian Zhang

China Manned Space Agency
Beijing, China
zhl_cms@163.com

Jiyuan Lv

China Astronaut Research and
Training Center
Beijing, China
lvjiyuan@139.com

Yusheng Liu*

State Key Lab of CAD/CG
Zhejiang University
Hangzhou, China
HangzhouHope System
Technology
Hangzhou, China
ysliu@cad.zju.edu.cn

Abstract—With the development and progress of Model-Based Systems Engineering (MBSE), it has gradually replaced the traditional text-based methods and has been widely used. Systems Modeling Language (SysML) is the de facto standard for MBSE approaches. However, there are various SysML-based system engineering modeling softwares, transformation of heterogeneous data from various softwares become an urgent need to meet. Here, we base on the XMI standard and the system engineering modeling software M-Design, achieve transformation from the XMI format file to the objects design according to SysML language. Firstly, we analyze a XMI format file structure, and secondly design a transformation solution according to analysis and SysML standard. After transformation, we compare result project and the original one to check consistency. With this method, heterogeneous data from different software can be communicated easily.

Keywords—MBSE, UML, SysML, XMI, Heterogeneous data transformation

I. INTRODUCTION

In Model-Based Systems Engineering (MBSE), a complex system can be described as a system architecture model. With this model, different departments can conduct unified development and optimization. In the development of complex system, MBSE not only has the advantages of easy-reading, high traceability and high-structured, but also supports simulation, verification and confirmation [1]. So now it has gradually replaced the traditional text-based methods and has been widely used.

Systems Modeling Language (SysML) is the de facto standard for MBSE approaches [2], it's a general-purpose modeling language, can support the specification, design, analysis, verification and validation of a wide range of systems.

At present, there are many SysML-based system engineering modeling software. International prevailing software include Magic Draw, Papyrus SysML, Ratchal Rhapsody, etc., while domestic software has M-Design [3]. They each have their own advantages, but still have insufficient. For example, Magic Draw is a powerful MBSE modeling tool. It has the characteristics of strict execution of SysML syntax and semantics, and integrates professional plugins for various fields, such as requirement arrangement tool DOORS and simulation tool Simulink. However, Magic Draw has complex UI, users need to be well trained to build a proper model. And its sequence diagram can't

fully handle signal syntax. Other softwares also have their own deficiencies. Papyrus SysML is small and free, but its function is not complete. Rational Rhapsody supports state machine diagram well, but the support for activity diagram and sequence diagram is weak. In addition, many softwares only support English, lack of translation to other language, it is not friendly to non-English users. Based on this status quo, users tend to choose the appropriate software according to their own needs. However, the save format of different software varies, which decreases interoperability of project between departments, causes hindrance to project collaboration. Hence, transformation of heterogeneous data from various softwares become an urgent need to meet.

Based on this need, OMG (Object Management Group) organization published XMI (XML Metadata interchange) standard, aimed to standardize the exchange format of metadata between different software [4]. In this study, we base on the XMI standard and the system engineering modeling software M-Design, achieve transformation from the XMI format file to the objects design according to SysML language. Any Project, no matter in which software it is built, can be converted to corresponding model objects and be opened in M-Design, as long as it can be exported to a XMI format file. By so, we achieve the interoperability between model projects from different software.

The paper is developed as follows: The next section provides background information and related works about XMI file transformation. Section 3 discusses the structure of XMI format file, and gives code examples, and a usable transformation solution is represented based on the analysis. Section 4 then shows the results of transformation, and checks the consistency of result project. The conclusion section sums up the meaning of this research and discusses further possible study direction.

II. RELATED WORKS

According to OMG organization's design, metadata information can be exchanged via XMI standard. XMI is a flexible and easy-parsing standard, it can be used for any metadata whose metamodel can be expressed in Meta-Object Facility (MOF).

Because XMI can ensure that SysML model information can be exchanged in consistency, and it enable easy interchange of metadata in distributed heterogeneous environments. So, since the release of XMI standard, many researchers have tried to

implement heterogeneous data interchange based on XMI standard [5, 6].

Some researchers have tried to transform UML use case diagram to XMI format file [7]. However, they only implement one way transformation. Based on XMI standard, they defined a mapping rule of use case diagram elements to XMI document, including use case, actor and relations. Then, some scholars hope to specify a unified process to transform UML model to XMI document [8, 9]. They tried to define generating rules of XML DTD and XML document, and used MSXML as application development library. But transformation based on custom XML DTD is rigid, it is impossible to handle the complex situation of the diverse SysML modeling software. Moreover, using third party library is not conducive to customization and subsequent maintenance of project.

In addition, due to the universality and ease of use of the XMI format, some researchers have tried to design a transformation method for a specific business field [10, 11]. Firstly, they use XMI technology to transform models into related XML files of specific field, and then use XSL mapper based on meta-model to transform these XML files to UML related XML files. Such method is effective in a certain field, but inadequate versatility and reuse.

In summary, so far, in the XMI-based heterogenous data transformation researches, most of them are only focus on UML model and diagram. Some of these studies have only implement parts of UML diagrams, lack a general transformation solution. For MBSE, a usable XMI-based transformation method for SysML metadata is urgently needed. SysML is defined as an extension of a subset of the UML, its semantic is more complicated, and has more diverse model and diagram definitions, so the difficulty of transformation is greatly increased. In addition, in previous studies, researches tend to use third-party libraries, so the transformation methods are always rigid. Hence, they are not capable of dealing with the diverse situations in the market, and not easy to customize, update and maintain.

III. XMI STRUCTURE ANALYSIS

A. XMI Semantic and File Structure Analysis

According to XMI standard, a XMI file exported from a complete SysML project should contain these following parts:

a) XMI Head Nodes: Like most XML files, information in head node includes document version, coding standard statement, software publisher and export version. Exporting software information can be stored in 'Documentation' node.

b) Model Node: A XMI format file must contain a model node, it takes a UML model element as a root node, and uses packaged element to construct the main body of the hierarchical architecture. Under the root node, there are 'packagedElement' nodes to store information. These packageable elements can be a UML package or a UML class, any subclasses that belong to the UML packageable element can be stored in these nodes. A new packaged element node can also be used as a root node of a subtree. The owned elements and owned properties of a packageable element can be store in the sub-nodes or attributes

inside the node tag. Moreover, for each model element, if it has owned comments, then these comments are saved in the owned comment node tag.

c) Nodes named by UML standard: The owned elements of a model element are not all saved in a packaged element node tag, there are some nodes named after UML standard. For example, a UML association node could have child nodes named 'memberEnd' and 'ownedEnd', which correspond to attributes of association defined by UML standard. The 'lowerValue' node under 'ownedEnd' tag can also find the corresponding definition in UML specification.

d) SysML stereotype nodes: As a expansion modeling language based on UML, SysML standard use stereotype to add most of its new definitions for systems engineering. Hence, there are also node for storing information about SysML. Stereotype node always has an attribute inside the node tag, which starts with 'base_', to indicate the base model ID which this stereotype applied. In other words, there is a basic model with this ID in the model node, and it takes this stereotype as 'applied stereotype'. If a stereotype node only stores the base model property, it indicates that there is no new defined attribute in this stereotype, or the value of new defined attribute is the default value.

e) Extension nodes: XMI format file support extension, its 'xmi:Extension' node tag can be used to save various kinds of extension information. One of the most important information is the definition of SysML stereotype. Extension nodes can contain definition of SysML stereotypes. When parsing a XMI format file, if we meet a SysML stereotype node, we can find the corresponding definition in extension nodes.

B. Transformation Process

Based on the analysis of XMI format file structure mentioned above, we can design a process to parse a XMI file (Fig. 1).

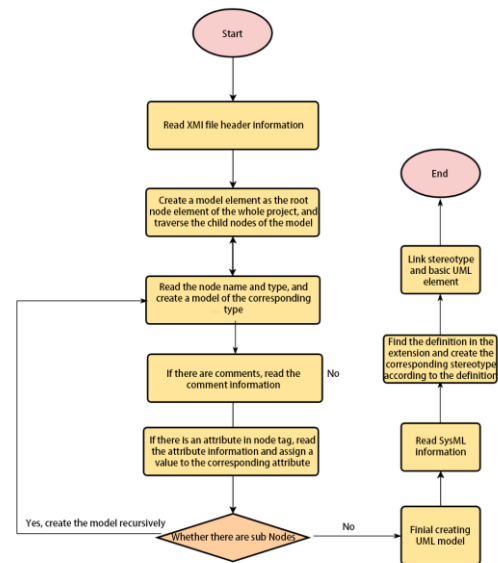


Fig. 1. Transformation process.

Before parsing model data, we need to read the head node to get some basic information, including namespace, publisher software, version control, etc. Then, we parse the <uml:Model> node tag, and use the data create a UML model element as the root element of the entire project. Afterwards, we recursively create elements under the entire model tree architecture. Firstly, we create child elements under the node model, and then recursively create child nodes of each child element. For every single model element node tag, the parsing steps are as follows:

1) Firstly, parse the name, id and type information inside the node tag, create a corresponding element object according to its type.

2) If the node has a <ownedComment> sub-node, create a comment model object according to the information in node, and link it to parent model element.

3) If there are other attributes information inside the node tag, parse the value of every attribute, find the corresponding element property defined by UML standard, and then set the value to the property. It is worth noting that there may be many properties in a UML model element, the corresponding data may store inside the node tag or in the child nodes. It would be too time consuming if we need to find out where the property information is store. So, for each property, we check if it exists inside the node tag, if not, then we go to next step to check if it is stored in child nodes.

4) If the node has child nodes, then recursively parse the child nodes. There are several situations need to handle. Firstly, some child nodes only store their reference id, more precise information is stored elsewhere. If we create element object every time we met an element node, we would create some objects with same id. So, here we maintain an object map to store the temporary object, every time before creating an object, we check the object map to avoid duplicate creation. Secondly, for a mature software like Magic Draw, it usually has tons of pre-defined elements saved in some profiles. When parsing these pre-defined elements, we can only get profile information and element id. So, it is essential to parsing related profile information before parsing a XMI file. Moreover, although most child nodes would have a identical name in element property definition, there still are some exception which need to handle manually.

5) If the node has extension nodes as child nodes, parse the extension information and create corresponding elements. For example, extension node can store graph information or primitive type information. Different software may use their own extension node format because it's relatively flexible, so parsing rules may need to customize.

6) After UML models are created, parse the contents of SysML stereotype nodes. According to type information in the node, create a stereotype object. If this stereotype is already defined in SysML standard, then we can parse the attribute values and set properties. If the stereotype is customized by project creator, we can find extension nodes which store the

stereotype definition. After getting a compact stereotype object, link it to its base UML element.

By this process, a model object based on XMI format project can be created. Because the completeness of information stored by XMI file, the result model can ensure its consistency.

IV. TRANSFORMATION RESULT

A complicated systems engineering project, after the transformation process mentioned above, can be transformed to another software, including its structure, data and diagrams. A complex internal block diagram can be well transformed(Fig. 2). As we can see in the graph, details in the diagram are well transformed, including association between requirement and block. Moreover, block's owned elements like properties, ports and associations are not missing. As for diagram presentation, the requirement's ID and name are well transformed. Text of requirement is also contact, however, due to the display rules of M-Design diagram, the requirement text is not display in the diagram.

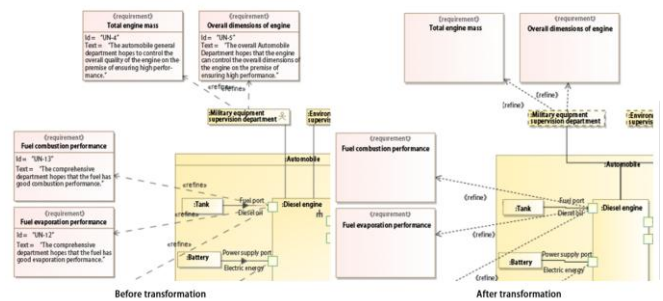


Fig. 2. Transformation result of a complex internal block diagram.

The transformation process also fit state machine diagram transformation well (Fig. 3). We can see that states, initial state, final state and transitions are all compactly transformed.

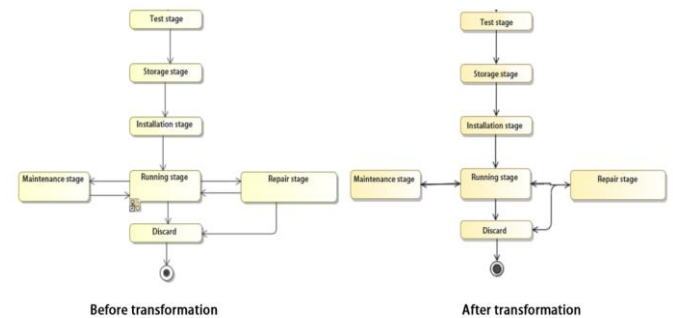


Fig. 3. Transformation result of state machine diagram.

In addition to the consistency between diagrams, in this study, we also check whether all elements in a model are completely transformed, especially in a large SysML project (Fig. 4). The result shows the integrity of transformed project, we can see that package hierarch structure is impact correct.

Apart from package hierarch structure consistency, we also checked the consistency of owned elements of every single model element (Fig. 5). In the transformed project, under the fuel supply subsystem package, no element is missing after

transformation. Including requirements, state machines, blocks and various relations.

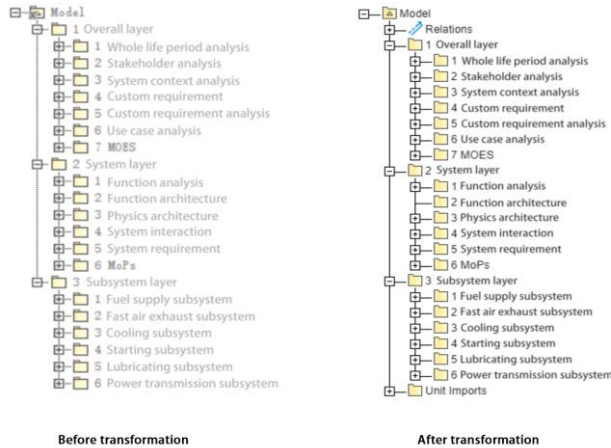


Fig. 4. Consistency of package hierarchy structure after transformation.

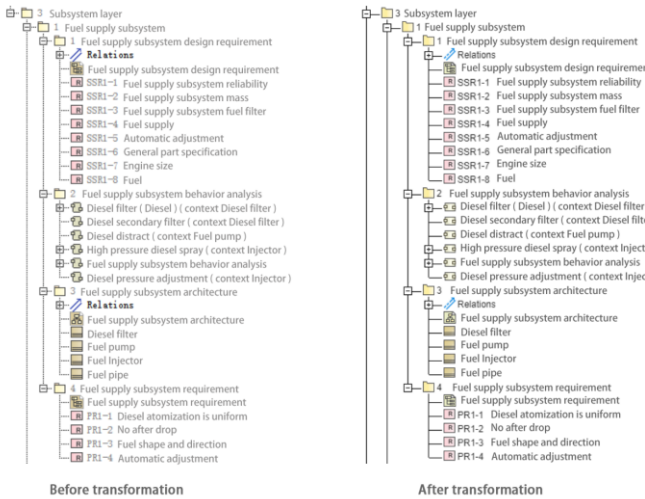


Fig. 5. Consistency of package's owned elements.

V. CONCLUSION

In this study, we studied UML, SysML and XMI standard specifications and defines. After having a comprehensive understanding of model and meta model definition of each standard, we deeply analyzed the format of XMI format document and how it corresponds to UML and SysML models.

With the development of MBSE, a lot of softwares are published, more and more projects are based on MBSE now. For an enormous systems engineering project, MBSE plays a crucial role of design, construction, maintenance and verification.

Nowadays, there are many systems engineering tools in the market, how to share project data between different software has become a huge problem. Our study implements a feasible method based on the analysis of XMI document, which can maintain the consistency of project data after transformation. It has great meaning for team communication and project sharing.

Based on the process this study proposed, we can easily parse a XMI format files from various software, the transformation result can also keep consistency. However, for a huge system engineering project, the integrity check is very time-consuming. In the future, consistency check methods for transformed files may need further study.

ACKNOWLEDGMENT

The present work has been in part supported by the National Key Research and Development Program of China (2018YFB1700901), National Science Foundation of China (61873236), and the Key project of NSF of Zhejiang Province (LZ21F020003).

REFERENCES

- [1] Cao Yue. Research on Modeling and Simulation Integrated Modeling and Simulation Integrated Research based on SYSML. [D]. Zhejiang University, 2011.
- [2] Delligatti, L. (2014). SysML distilled: A brief guide to the systems modeling language (Vol. 2). Addison-Wesley.
- [3] Xu Zihe. The development and tracking system of complex product demand for MBSE [D]. Zhejiang University, 2017.
- [4] Shan Dexiang. Research and application based on XMI -based metadata exchange technology [D]. Beijing University of Posts and Telecommunications, 2006.
- [5] Li Wen, Xie Fuwen, Zou Daoming. XML data exchange technology [J]. Computer and modernization, 2008 (01): 91-93.5
- [6] Li Junhui, Zhou Mingquan, Geng Guohua, Zhang Jingxi. XML's application research in heterogeneous data integration [J]. Computer application, 2002 (09): 10-12.
- [7] Shen Yuncheng, Yan Changqin. UML use diagram to XMI mapping method Study [J]. Journal of Guiyang College (Natural Science Edition), 2015,10 (02): 10-14.Doi: 10.16856/J.CNKI. 52-1142/n.2015.02.004.
- [8] Li Haifeng, Jin Hao, Wu Jieyi. Use XMI standards to implement the conversion of UML model to XML [J]. Manufacturing automation, 2003 (04): 12-16.
- [9] He Zhuozhu, Liu Zhiyong, Li Lu, Li Changming, Zhang Lin. Comparison of XML Analysis Methods in the Data Transformation of heterogeneous text data [J]. Computer engineering, 2020,46 (07): 286-293+299.DOI: 10.19678/j.issn.1000-3428.0054925.
- [10] Xu Jianqiang, Zhu Yong. Based on XMI standards to realize UML to XML conversion [J]. Technology information, 2010 (19): 462+571.
- [11] Zhu Qin, Lu Jianxin, Chen Jihong. XML-based heterogeneous data exchange technology and its Java implementation [J]. Computer application and software, 2004 (11): 52-53+125.